

CSA17 · ARTIFICIAL INTELLIGENCE · CO5 TECHNICAL PRESENTATION

Multi-Agent System for Disaster Management

Coordinating sensor, decision, and communication agents for faster disaster detection and response

Presented by: Athavan K | Reg No: 192524036



Disasters Move Faster Than Traditional Response Systems

72 hrs

Golden window for effective search & rescue
before survival odds fall sharply

3–6 hrs

Typical delay before centralized command
centers get a full situational picture

40%+

Response effort often wasted on duplicated
or poorly coordinated tasks

Why single-agent systems fall short

- One central controller becomes a bottleneck and a single point of failure when networks or infrastructure go down
- Disaster zones generate high-volume, high-velocity data (sensors, drones, field reports) that no single process can evaluate in real time
- Rescue, medical, and logistics tasks need parallel, localized decisions — not a queue behind one decision-maker

Objectives of This Presentation



Define the Architecture

Establish what a multi-agent system is and why it fits disaster response.



Detail Each Agent Role

Break down sensor, decision, and communication agents and how they interact.



Explain Coordination

Show the negotiation and data-sharing strategies that keep agents aligned.



Cover Tools & Scale

Map the technology stack that makes real-time, distributed decisions possible.

What Is a Multi-Agent System?

A Multi-Agent System (MAS) is a network of autonomous, intelligent agents that perceive their environment, act independently, and coordinate with one another to achieve a shared goal — without relying on one central controller.

Single-Agent vs. Multi-Agent for Disaster Response

	Single-Agent	Multi-Agent
Fault tolerance	Low — one failure halts response	High — agents reassign tasks
Speed at scale	Degrades with data volume	Scales via parallel processing
Local context	Global view only	Zone-level, real-time context

KEY PROPERTIES

Autonomy

Each agent senses and acts on its own, without step-by-step instructions from a human or central node

Local Perception

Agents work off their own sensor/data feed, not a single global view — reducing latency

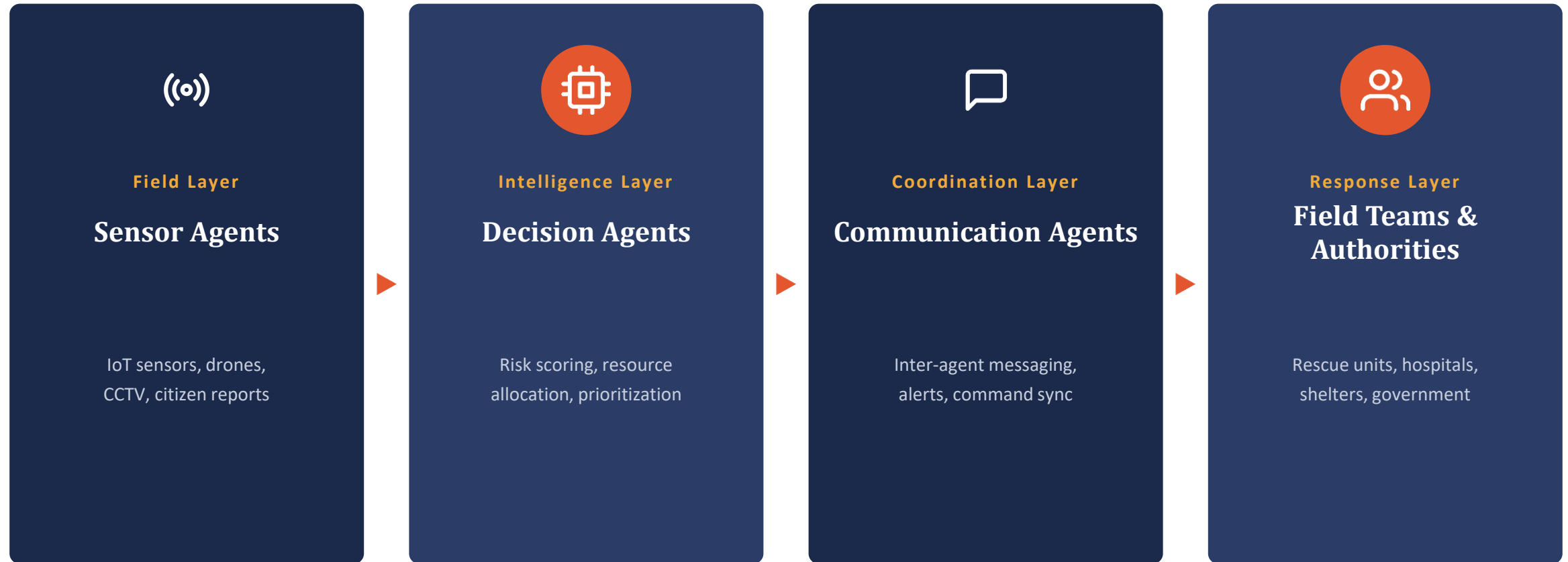
Social Ability

Agents exchange messages and negotiate to align on a shared response strategy

Reactivity + Proactivity

Agents respond to sudden events (a collapsed bridge) while pursuing longer-term goals (route survivors to shelters)

End-to-End System Architecture



Data and alerts flow left to right; feedback and re-tasking flow right to left as conditions change.

Sensor Agents — Sensing the Disaster in Real Time



*"The eyes and ears of the system" —
continuous environmental monitoring at the
edge*

Core Responsibilities

- Continuously stream data from IoT sensors, drone feeds, CCTV, weather stations, and citizen SMS/app reports
- Perform lightweight edge filtering to flag anomalies (flood level rise, seismic spike, smoke detection) before forwarding
- Tag every reading with geolocation and timestamp so downstream agents can prioritize by zone and urgency
- Publish structured events to a streaming layer (e.g., PySpark Structured Streaming) instead of a static database

EXAMPLE

During a flash flood, river-level sensor agents detect a 1.2m rise in 10 minutes and immediately publish a high-severity event tagged to the downstream district.

Decision Agents — Turning Data Into Action



"The reasoning core" — converts raw signals into ranked, actionable response plans

Core Responsibilities

- Consume the sensor-agent event stream and compute a real-time risk/severity score per affected zone
- Apply rule-based and learned decision logic to rank competing needs (medical vs. evacuation vs. infrastructure)
- Allocate scarce resources — rescue teams, ambulances, relief supplies — using optimization over live constraints
- Continuously re-plan as new sensor data arrives, rather than committing to a single static response plan

EXAMPLE

Given three simultaneous alerts, the decision agent ranks a collapsed-building zone above a minor road blockage and dispatches the nearest available rescue unit first.

Communication Agents — Keeping Everyone in Sync



"The connective tissue" — reliable message passing between agents and human responders

Core Responsibilities

- Route decisions and alerts between sensor agents, decision agents, field teams, and central command
- Maintain a shared, consistent situational picture across all agents despite network delays or partial outages
- Translate structured agent output into human-readable alerts (SMS, dashboards, radio dispatch instructions)
- Handle retries and failover messaging when a node or communication link goes down mid-response

EXAMPLE

When cellular towers in one district fail, the communication agent reroutes updates through a mesh/satellite fallback so field teams never lose command instructions.

Coordination Strategies Between Agents



Contract-Net Protocol

Decision agents broadcast a task; sensor/field agents "bid" based on proximity and capacity, and the best-fit agent wins the assignment.



Shared Blackboard

A common data space (backed by a distributed store) where all agents read and write the latest situational state to avoid conflicting actions.



Goal Negotiation

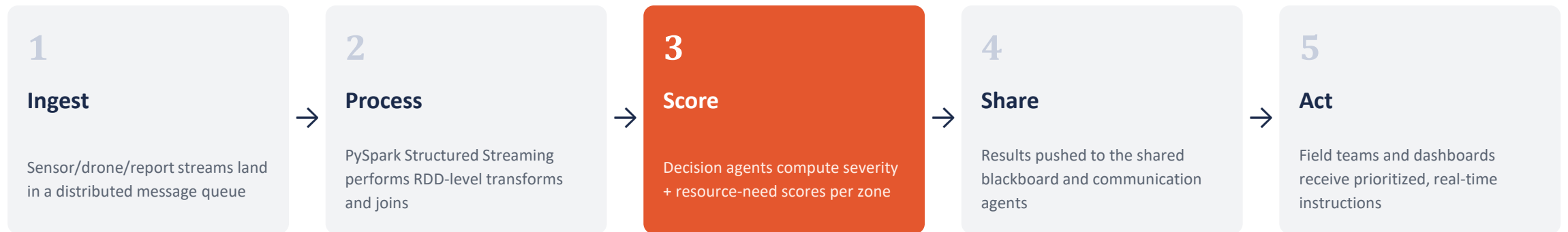
Agents with competing priorities (e.g., two zones needing the same ambulance) negotiate using priority scores until consensus is reached.



Event-Driven Sync

Agents subscribe to real-time event streams and react instantly to state changes, rather than polling on a fixed schedule.

Real-Time Data Sharing at Scale



Why streaming instead of batch processing

- Disaster conditions change in minutes — a batch job run hourly can dispatch teams to a location that has already changed
- RDD-based distributed processing lets the system handle spikes in sensor volume (e.g., during a widening flood) without falling behind
- In-memory processing keeps agent-to-agent latency low enough for near-real-time coordination across zones

Technology Stack



PySpark / RDDs

Distributed processing of high-volume sensor and report streams



Structured Streaming

Real-time windowed aggregation for severity scoring



Message Broker

Kafka-style pub/sub for inter-agent event delivery



GIS / Geo-tagging

Zone-level mapping of alerts, resources, and routes



Agent Framework

Rule-based + ML decision logic per agent role



Fault-Tolerant Store

Replicated blackboard state so no single node loss stalls response

Challenges & Future Scope

CURRENT CHALLENGES

- Network connectivity loss in the worst-hit zones can isolate sensor and communication agents
- Conflicting priorities between agents (e.g., two zones needing the same resource) require robust negotiation logic
- Data quality from citizen reports and low-cost sensors can be noisy or inconsistent
- Coordinating across multiple agencies with different data formats and protocols

FUTURE SCOPE

- Reinforcement learning agents that improve resource allocation from past disaster response outcomes
- Satellite and drone-swarm integration for continuous coverage when ground infrastructure fails
- Predictive agents that forecast disaster escalation (e.g., flood spread) rather than only reacting to it
- Standardized inter-agency protocols to plug government, NGO, and private response systems into one MAS

CONCLUSION

Coordinated Agents, Faster Response, More Lives Saved

1

Sensor, decision, and communication agents together replace a single fragile control point with a resilient, distributed response network

2

PySpark-based streaming and RDD processing let the system scale to real disaster data volumes without losing real-time responsiveness

3

Coordination protocols (contract-net, shared blackboard, negotiation) keep dozens of autonomous agents working toward one shared goal

Thank you — Questions & Discussion